Project 2: Detecting Anomalies in Credit Card Transactions (CUSUM Algorithm)

Project Objective

The goal of this project is to develop a program that **detects abnormal credit card transactions in a continuous data stream**. The system will use the **CUSUM (Cumulative Sum)** algorithm — a classic statistical method designed to identify **sudden or gradual changes in data patterns**. Students will learn how to apply stream-processing techniques to identify anomalies efficiently, without storing large historical datasets.

CUSUM is often used in **finance**, **manufacturing**, and **network monitoring** to detect significant deviations from normal behavior as soon as they occur.

How the CUSUM Algorithm Works

CUSUM (Cumulative Sum Control Chart) is based on tracking the **cumulative deviation of observed data** from a reference mean (expected value). If the cumulative deviation grows too large, it indicates a possible anomaly.

In simpler terms, CUSUM answers the question:

> "Has the process changed significantly compared to its past average?"

Main Idea

For each new transaction value (x_t), CUSUM updates two variables:

- $ S_t^+ $: detects **positive changes** (sudden increases)
- $ S_t^- $: detects **negative changes** (sudden decreases)

They are updated using the formulas:

$S_t^+ = \max(0, S_{t-1}^+ + x_t - \mu_0 - k)$

$S_t^- = \max(0, S_{t-1}^- - x_t + \mu_0 - k)$

Where:

- $ \mu_0 $ — reference (expected) average transaction value
- $ k $ — tolerance constant (how much change is allowed before we react)
- $ h $ — decision threshold (if $ S_t^+ $ or $ S_t^- $ > $ h $, an anomaly is detected)

When either cumulative sum exceeds the threshold, it signals that the process has shifted — i.e., an anomaly or suspicious transaction may have occurred.

Intuitive Explanation

Imagine monitoring the average daily spending of a customer. Normally, their transactions hover around $50. CUSUM will gradually accumulate deviations above or below this mean:

- If transactions stay close to $50 → cumulative sum resets to 0 (no anomaly).
- If a sudden jump occurs (e.g., multiple $500 payments) → deviations accumulate → threshold exceeded → **alarm triggered**.

Example

Let's say the expected mean $ \mu_0 $ = 50, tolerance $k = 5$, threshold $h = 20$.

Transaction (USD)	Deviation	Cumulative +	Cumulative -	Alert
45	-5	0	0	–
52	+2	0	0	–
60	+10	5	0	–
55	+5	5	0	–
80	+30	30	0	⚠ **Anomaly!**

After a few normal values, the jump to 80 causes $ S_t^+ $ to exceed the threshold — an anomaly is detected.

Assignment

1. **Write a program** (in Python, Java, or JavaScript) that:
    - Reads a stream or file of simulated transaction values (e.g., `transactions.csv`)
    - Continuously updates the CUSUM values $ S_t^+ $ and $ S_t^- $
    - Flags transactions that exceed the detection threshold
2. **Implement the CUSUM algorithm** with parameters:
    - `mean` (μ_0): baseline transaction value
    - `k`: tolerance constant
    - `h`: decision threshold These should be configurable (e.g., via command line or config file).
3. **Test your program** with a dataset that contains both normal and anomalous transactions.
4. **Output**:
    - A list or visualization of detected anomalies (e.g., timestamp, amount, deviation)
    - Summary statistics (number of anomalies, average deviation, etc.)

Results and Report

Your report should include:

- A short explanation of the **CUSUM principle**
- A description of your **dataset** (normal vs. anomalous transactions)
- Discussion of how parameter tuning affects detection sensitivity
- Conclusions about the algorithm's effectiveness in real-time transaction monitoring